

Dolphus2 / exam_utils.py

Secret



Created now

[Code](#) [Revisions](#) 1

exam_utils.py

exam_utils.py

```
1  from __future__ import annotations
2
3  from time import time
4  import matplotlib.pyplot as plt
5  import jax
6  import jax.numpy as jnp
7  import numpy as np
8  import jax.scipy.stats as jstats
9  from jax import random, hessian, value_and_grad, vmap, jit, grad
10 from jax.scipy.special import gammaln
11 from jax.typing import ArrayLike
12 import seaborn as snb
13 from dataclasses import dataclass
14 from scipy.optimize import minimize
15 from scipy.stats import norm
16 from typing import Any, Optional, Sequence, Union
17 from collections.abc import Callable
18 from matplotlib.axes import Axes
19 from matplotlib.figure import Figure
20
21 from mpl_toolkits.axes_grid1 import make_axes_locatable
22
23 from jax.scipy.stats import multivariate_normal as mvn
24
25 from jax import config
26 config.update("jax_enable_x64", True)
27
28 snb.set_style('darkgrid')
29 snb.set_theme(font_scale=1.25)
30 colors = snb.color_palette()
31
32 # ===== Utility Functions =====
33
34 # Calculate empirical confidence interval for a list of quantiles.
35 #jnp.percentile(random.normal(random.key(0), (1000,)), jnp.array([2.5, 97.5]))
36
```

```

37 class Grid2D(object):
38     """ helper class for evaluating the function func on the grid defined by (alpha, be
39
40     def __init__(self, alphas: jax.Array, betas: jax.Array,
41                 func: Callable[[ArrayLike, ArrayLike], jax.Array], name: str = "Grid2D
42         self.alphas = alphas
43         self.betas = betas
44         self.grid_size = (len(self.alphas), len(self.betas))
45         self.alpha_grid, self.beta_grid = jnp.meshgrid(alphas, betas, indexing='ij')
46         self.func = func
47         self.name = name
48
49         # evaluate function on each grid point
50         self.values = self.func(self.alpha_grid[:, :, None], self.beta_grid[:, :, None]
51
52     def plot_contours(self, ax: Axes, color: str = 'b', num_contours: int = 10,
53                     f: Callable = lambda x: x, alpha: float = 1.0, title: Optional[str
54         ax.contour(self.alphas, self.betas, f(self.values).T, num_contours, colors=colo
55         ax.set(xlabel='$\\alpha$', ylabel='$\\beta$')
56         ax.set_title(self.name, fontweight='bold')
57
58     @property
59     def argmax(self) -> tuple[jax.Array, jax.Array]: # approximation for small 2D probl
60         idx = jnp.argmax(self.values)
61         alpha_idx, beta_idx = jnp.unravel_index(idx, self.grid_size)
62         return self.alphas[alpha_idx], self.betas[beta_idx]
63
64 class GridApproximation2D(Grid2D):
65     """Normalized grid approximation of a 2D posterior distribution.
66
67     Evaluates log_joint on a (alpha, beta) grid, normalizes to a proper
68     probability distribution, and provides marginals, expectation, and sampling.
69
70     Usage::
71         grid = GridApproximation2D(alphas, betas, model.log_joint)
72         alpha_samples, beta_samples = grid.sample(key, num_samples=500)
73         E_theta = grid.compute_expectation(lambda a, b: a + b)
74         grid.visualize(ax)
75     """
76
77     def __init__(self, alphas: jax.Array, betas: jax.Array,
78                 log_joint: Callable[[ArrayLike, ArrayLike], jax.Array],
79                 threshold: float = 1e-8,
80                 name: str = "GridApproximation2D") -> None:
81         Grid2D.__init__(self, alphas, betas, log_joint, name)
82         self.threshold = threshold
83         self._prep_approximation()
84         self._compute_marginals()

```

```

85         self._sanity_check()
86
87     def _prep_approximation(self) -> None:
88         log_vals = self.values - jnp.max(self.values)
89         tilde = jnp.exp(log_vals)
90         self.probabilities_grid: jax.Array = tilde / jnp.sum(tilde)
91         self.alphas_flat: jax.Array = self.alpha_grid.flatten()
92         self.betas_flat: jax.Array = self.beta_grid.flatten()
93         self.num_outcomes: int = int(len(self.alphas_flat))
94         self.probabilities_flat: jax.Array = self.probabilities_grid.flatten()
95
96     def _compute_marginals(self) -> None:
97         self.pi_alpha: jax.Array = self.probabilities_grid.sum(axis=1) # (num_alpha,)
98         self.pi_beta: jax.Array = self.probabilities_grid.sum(axis=0) # (num_beta,)
99
100    def _sanity_check(self) -> None:
101        assert self.probabilities_grid.shape == self.grid_size
102        assert jnp.all(self.probabilities_grid >= 0)
103        assert jnp.allclose(self.probabilities_grid.sum(), 1.0)
104
105    def compute_expectation(self, f: Callable[[ArrayLike, ArrayLike], jax.Array]) -> jax.Array:
106        """E_q[f(alpha, beta)] under the grid approximation."""
107        return jnp.sum(f(self.alphas_flat, self.betas_flat) * self.probabilities_flat,
108                        axis=-1)
109
110    def sample(self, key: jax.Array, num_samples: int = 1) -> tuple[jax.Array, jax.Array]:
111        """Draw samples of (alpha, beta) from the grid approximation.
112
113        returns:
114            alpha_samples, beta_samples -- each shape (num_samples, 1)
115        """
116        idx = random.choice(key, jnp.arange(self.num_outcomes),
117                            p=self.probabilities_flat, shape=(num_samples, 1))
118        return self.alphas_flat[idx], self.betas_flat[idx]
119
120    def visualize(self, ax: Axes, scaling: float = 8000,
121                 title: str = '') -> None:
122        """Scatter plot of grid points; point area proportional to probability mass."""
123        mask = self.probabilities_flat > self.threshold
124        ax.scatter(self.alphas_flat[mask], self.betas_flat[mask],
125                  scaling * self.probabilities_flat[mask], label='$\\pi_{ij}$')
126        ax.set(xlabel='$\\alpha$', ylabel='$\\beta$')
127        ax.set_title(title or self.name, fontweight='bold')
128
129    def plot_grid_marginals(grid: GridApproximation2D,
130                           param_names: Optional[tuple[str, str]] = None,
131                           figsize: Optional[tuple[float, float]] = None,
132                           ) -> tuple[Figure, np.ndarray]:

```

```

133     """Bar charts of the marginal distributions of a GridApproximation2D.
134
135     arguments:
136         grid          -- fitted GridApproximation2D
137         param_names   -- (alpha_name, beta_name); defaults to ('α', 'β')
138         figsize       -- figure size
139
140     returns:
141         fig, axes     -- 1x2 subplot array
142     """
143     a_name, b_name = param_names or ('$\\alpha$', '$\\beta$')
144     fig, axes = plt.subplots(1, 2, figsize=figsize or (12, 4))
145     axes[0].bar(np.array(grid.alphas), np.array(grid.pi_alpha),
146                width=float(grid.alphas[1] - grid.alphas[0]))
147     axes[0].set(xlabel=a_name, ylabel=f'$q({a_name})$', title=f'Marginal of {a_name}')
148     axes[1].bar(np.array(grid.betas), np.array(grid.pi_beta),
149                width=float(grid.betas[1] - grid.betas[0]))
150     axes[1].set(xlabel=b_name, ylabel=f'$q({b_name})$', title=f'Marginal of {b_name}')
151     fig.tight_layout()
152     return fig, axes
153
154
155 # ===== Activation Functions =====
156
157 def sigmoid(x: ArrayLike) -> jax.Array:
158     return 1.0 / (1.0 + jnp.exp(-x))
159
160 def softplus(x: ArrayLike) -> jax.Array:
161     return jnp.log(1.0 + jnp.exp(x))
162
163 def relu(x: ArrayLike) -> jax.Array:
164     return jnp.maximum(0.0, x)
165
166 def softmax(x: ArrayLike) -> jax.Array:
167     e_x = jnp.exp(x - jnp.max(x))
168     return e_x / e_x.sum()
169
170 # ===== Probability Distributions =====
171
172 def probit(x: ArrayLike) -> np.ndarray:
173     """Gaussian CDF – used as a link function for binary classification."""
174     return norm.cdf(x)
175
176 log_npdf = lambda x, m, v: -(x-m)**2/(2*v) - 0.5*jnp.log(2*jnp.pi*v) # v is the variance
177 npdf = lambda x, m, v: jnp.exp(log_npdf(x, m, v))
178
179 def gaussian_logpdf(x: jax.Array, mu: float, sigma: float) -> jax.Array: # sigma is the
180     return -0.5 * jnp.log(2 * jnp.pi) - jnp.log(sigma) - 0.5 * ((x - mu) / sigma) ** 2

```

```
181
182 def gaussian_pdf(x: ArrayLike, mu: float | ArrayLike, sigma: float | ArrayLike) -> jax.
183     return jnp.exp(gaussian_logpdf(x, mu, sigma))
184
185 def bernoulli_logpmf(y: ArrayLike, p: ArrayLike) -> jax.Array:
186     """Log PMF of Bernoulli: y in {0, 1}, p in (0, 1)."""
187     return y * jnp.log(p) + (1 - y) * jnp.log(1 - p)
188
189 # Multivariate Normal
190 def mvn_logpdf(x: ArrayLike, mu: ArrayLike, Sigma: ArrayLike) -> jax.Array:
191     """Log PDF of multivariate normal N(mu, Sigma).
192
193     arguments:
194         x      -- D-vector
195         mu     -- D-vector mean
196         Sigma  -- DxD covariance matrix
197     """
198     return mvn.logpdf(x, mu, Sigma)
199
200 def mvn_pdf(x: ArrayLike, mu: ArrayLike, Sigma: ArrayLike) -> jax.Array:
201     return jnp.exp(mvn_logpdf(x, mu, Sigma))
202
203 def mvn_sample(key: jax.Array, mu: ArrayLike, Sigma: ArrayLike,
204               shape: tuple[int, ...] = ()) -> jax.Array:
205     """Sample from N(mu, Sigma). shape gives the batch shape of samples."""
206     return random.multivariate_normal(key, mu, Sigma, shape=shape)
207
208 # TODO: Add function to compute entropy of normal distribution
209
210 # Gamma – parameterised by shape (a) and scale. Mean = a * scale.
211 def gamma_logpdf(x: ArrayLike, a: float | ArrayLike, scale: float | ArrayLike = 1.0) ->
212     """Log PDF of Gamma(a, scale). Mean = a * scale, variance = a * scale**2."""
213     return jstats.gamma.logpdf(x, a=a, scale=scale)
214
215 def gamma_pdf(x: ArrayLike, a: float | ArrayLike, scale: float | ArrayLike = 1.0) -> ja
216     return jnp.exp(gamma_logpdf(x, a, scale))
217
218 def gamma_sample(key: jax.Array, a: float | ArrayLike, scale: float | ArrayLike = 1.0,
219                 shape: tuple[int, ...] = ()) -> jax.Array:
220     return random.gamma(key, a, shape=shape) * scale
221
222 # Beta
223 def beta_logpdf(x: ArrayLike, a: float | ArrayLike, b: float | ArrayLike) -> jax.Array:
224     """Log PDF of Beta(a, b). Support x in (0, 1)."""
225     return jstats.beta.logpdf(x, a=a, b=b)
226
227 def beta_pdf(x: ArrayLike, a: float | ArrayLike, b: float | ArrayLike) -> jax.Array:
228     return jnp.exp(beta_logpdf(x, a, b))
```

```
229
230 def beta_sample(key: jax.Array, a: float | ArrayLike, b: float | ArrayLike,
231                 shape: tuple[int, ...] = ()) -> jax.Array:
232     return random.beta(key, a, b, shape=shape)
233
234 # Poisson
235 def poisson_logpmf(k: ArrayLike, lam: float | ArrayLike) -> jax.Array:
236     """Log PMF of Poisson(lam). k must be a non-negative integer."""
237     return jstats.poisson.logpmf(k, mu=lam)
238
239 def poisson_pmf(k: ArrayLike, lam: float | ArrayLike) -> jax.Array:
240     return jnp.exp(poisson_logpmf(k, lam))
241
242 def poisson_sample(key: jax.Array, lam: float | ArrayLike, shape: tuple[int, ...] = ())
243     return random.poisson(key, lam=lam, shape=shape)
244
245 # Binomial
246 def binomial_logpmf(k: ArrayLike, n: int, p: float | ArrayLike) -> jax.Array:
247     """Log PMF of Binomial(n, p).
248
249     arguments:
250         k -- number of successes (non-negative integer)
251         n -- number of trials (positive integer)
252         p -- success probability in (0, 1)
253     """
254     log_coeff = gammaln(n + 1) - gammaln(k + 1) - gammaln(n - k + 1)
255     return log_coeff + k * jnp.log(p) + (n - k) * jnp.log(1 - p)
256
257 def binomial_pmf(k: ArrayLike, n: int, p: float | ArrayLike) -> jax.Array:
258     return jnp.exp(binomial_logpmf(k, n, p))
259
260 def binomial_sample(key: jax.Array, n: int, p: float | ArrayLike,
261                     shape: tuple[int, ...] = ()) -> jax.Array:
262     """Sample from Binomial(n, p) by summing n Bernoulli trials."""
263     return jnp.sum(random.bernoulli(key, p, shape=(*shape, n)), axis=-1)
264
265 # Dirichlet
266 def dirichlet_logpdf(x: ArrayLike, alpha: ArrayLike) -> jax.Array:
267     """Log PDF of Dirichlet(alpha). x must be a probability vector summing to 1.
268
269     arguments:
270         x      -- K-vector in the probability simplex
271         alpha  -- K-vector of concentration parameters (all positive)
272     """
273     return jstats.dirichlet.logpdf(x, alpha=alpha)
274
275 def dirichlet_pdf(x: ArrayLike, alpha: ArrayLike) -> jax.Array:
276     return jnp.exp(dirichlet_logpdf(x, alpha))
```

```

277
278 def dirichlet_sample(key: jax.Array, alpha: ArrayLike,
279                     shape: tuple[int, ...] = ()) -> jax.Array:
280     return random.dirichlet(key, alpha, shape=shape)
281
282 # ===== Bayesian Linear Regression =====
283
284 def compute_posterior_w(Phi: ArrayLike, t: ArrayLike,
285                       alpha: float | ArrayLike, beta: float | ArrayLike) -> tuple[jax
286     """
287     Computes posterior p(w|t) of a linear Gaussian system
288
289     Arguments:
290         Phi:    NxM matrix of N observations with M features
291         t:      N-vector of targets
292         alpha:  prior precision (scalar)
293         beta:   likelihood precision (scalar)
294
295     Returns:
296         m:      M-vector posterior mean
297         S:      MxM posterior covariance
298     """
299     N, M = Phi.shape
300     A = alpha * jnp.identity(M) + beta * Phi.T @ Phi
301     m = beta * jnp.linalg.solve(A, Phi.T) @ t
302     S = jnp.linalg.inv(A)
303     return m, S
304
305 def marginal_likelihood(Phi: ArrayLike, t: ArrayLike,
306                       alpha: float | ArrayLike, beta: float | ArrayLike) -> jax.Array
307     """Computes log marginal likelihood of a linear Gaussian system.
308
309     Arguments:
310         Phi:    NxM design matrix
311         t:      N-vector of targets
312         alpha:  prior precision (scalar)
313         beta:   likelihood precision (scalar)
314
315     Returns:
316         log_Z:  log marginal likelihood (scalar)
317     """
318     N, M = Phi.shape
319     m, S = compute_posterior_w(Phi, t, alpha, beta)
320     Em = beta / 2 * jnp.sum((t - Phi @ m) ** 2) + alpha / 2 * jnp.sum(m ** 2)
321     A = alpha * jnp.identity(M) + beta * Phi.T @ Phi
322     return M / 2 * jnp.log(alpha) + N / 2 * jnp.log(beta) - Em - 0.5 * jnp.linalg.slogd
323
324 # ===== Laplace Approximation =====

```

```

325
326 def laplace_approximation(log_target: Callable[[ArrayLike], jax.Array],
327                           w0: ArrayLike) -> tuple[jax.Array, jax.Array]:
328     """Computes the Laplace approximation of a log-target density.
329
330     Arguments:
331         log_target: callable, log of the (unnormalized) target density
332         w0:         initial parameter vector (1D array)
333
334     Returns:
335         m: MAP estimate
336         S: covariance matrix from the Hessian at the MAP
337     """
338     obj = lambda w: -log_target(w)
339     result = minimize(value_and_grad(obj), w0, jac=True)
340     if not result.success:
341         print('Warning: Laplace approximation optimization failed!')
342     m = jnp.array(result.x)
343     S = jnp.linalg.inv(hessian(obj)(m))
344     return m, S
345
346 # ===== MCMC and HMC =====
347
348 def metropolis(log_target: Callable[[ArrayLike], jax.Array],
349               num_params: int,
350               tau: float,
351               num_iter: int,
352               theta_init: Optional[ArrayLike] = None,
353               seed: int = 0,
354               verbose: bool = True) -> jax.Array:
355     """Metropolis-Hastings sampler with isotropic Gaussian proposal.
356
357     arguments:
358         log_target: callable log_target(theta) where theta has shape (num_params,). Re
359         num_params: dimension of the parameter space
360         tau:         std. dev. of the Gaussian proposal distribution
361         num_iter:    number of iterations
362         theta_init:  initial parameter vector, shape (num_params,). Defaults to zeros.
363         seed:        random seed
364
365     returns:
366         thetas:      MCMC samples, shape (num_iter+1, num_params)
367     """
368     key = random.PRNGKey(seed)
369     theta = jnp.zeros(num_params) if theta_init is None else jnp.asarray(theta_init, dt
370     log_p = log_target(theta)
371     thetas = [theta]
372     accept_count = 0

```



```

373     t0 = time()
374     for _ in range(num_iter):
375         key, key_prop, key_acc = random.split(key, 3)
376         theta_star = theta + tau * random.normal(key_prop, shape=(num_params,))
377         log_p_star = log_target(theta_star)
378         if jnp.log(random.uniform(key_acc)) < jnp.minimum(0.0, log_p_star - log_p): # m
379             theta, log_p = theta_star, log_p_star
380             accept_count += 1
381             thetas.append(theta)
382
383     if verbose:
384         print(f'MCMC done: accept_rate={accept_count/num_iter:.3f}   t={time()-t0:.1f}s')
385
386     return jnp.array(thetas)
387
388
389 def _build_hmc_fns(log_target: Callable[[ArrayLike], jax.Array],
390                   ) -> tuple[Callable, Callable, Callable]:
391     """Build jit-compiled Hamiltonian, potential energy and its gradient from log_target
392
393     arguments:
394         log_target: callable log_target(theta) where theta has shape (D,). Returns sca
395
396     returns:
397         hamiltonian:    H(theta, nu) = -log_target(theta) + 0.5*||nu||^2
398         potential:      U(theta)      = -log_target(theta)
399         grad_potential: ∇U(theta)
400     """
401     @jit
402     def potential(theta: jax.Array) -> jax.Array:
403         return -log_target(theta)
404
405     grad_potential = jit(grad(potential))
406
407     @jit
408     def hamiltonian(theta: jax.Array, nu: jax.Array) -> jax.Array:
409         return potential(theta) + 0.5 * jnp.dot(nu, nu)
410
411     return hamiltonian, potential, grad_potential
412
413
414 def leapfrog(theta: jax.Array,
415             nu: jax.Array,
416             grad_potential: Callable[[ArrayLike], jax.Array],
417             num_steps: int,
418             step_size: float) -> tuple[jax.Array, jax.Array]:
419     """Leapfrog integrator for Hamiltonian dynamics.
420

```

```

421     arguments:
422         theta:          current position, shape (D,)
423         nu:             current momentum, shape (D,)
424         grad_potential: callable  $\nabla U(\theta)$ , shape (D,)  $\rightarrow$  (D,)
425         num_steps:      number of leapfrog steps L
426         step_size:      step size  $\epsilon$ 
427
428     returns:
429         theta: updated position, shape (D,)
430         nu:    updated momentum, shape (D,)
431     """
432     for _ in range(num_steps):
433         nu = nu - 0.5 * step_size * grad_potential(theta)
434         theta = theta + step_size * nu
435         nu = nu - 0.5 * step_size * grad_potential(theta)
436     return theta, nu
437
438
439 def HMC(log_target: Callable[[ArrayLike], jax.Array],
440        num_iterations: int,
441        theta0: ArrayLike,
442        num_leapfrog_steps: int = 10,
443        step_size: float = 0.1,
444        seed: int = 0,
445        verbose: bool = True) -> jax.Array:
446     """Hamiltonian Monte Carlo sampler.
447
448     arguments:
449         log_target:      callable log_target(theta) where theta has shape (D,). Retu
450         num_iterations:  number of HMC iterations
451         theta0:          initial position, shape (D,) or (1, D)
452         num_leapfrog_steps: number of leapfrog steps L per iteration
453         step_size:       leapfrog step size  $\epsilon$ 
454         seed:            random seed
455         verbose:         print progress and acceptance rate
456
457     returns:
458         thetas:          samples, shape (num_iterations+1, D)
459     """
460     hamiltonian, _, grad_potential = _build_hmc_fns(log_target)
461     key = random.PRNGKey(seed)
462     theta = jnp.asarray(theta0, dtype=float).ravel()
463     D = len(theta)
464     thetas = [theta]
465     accept_count = 0
466     t0 = time()
467
468     for i in range(num_iterations):

```

```

469     key, key_nu, key_acc = random.split(key, 3)
470     nu = random.normal(key_nu, shape=(D,))
471     theta_star, nu_star = leapfrog(theta, nu, grad_potential, num_leapfrog_steps, s
472     log_accept = jnp.minimum(0.0, -hamiltonian(theta_star, nu_star) + hamiltonian(t
473     if jnp.log(random.uniform(key_acc)) < log_accept:
474         theta = theta_star
475         accept_count += 1
476     thetas.append(theta)
477     if verbose and (i + 1) % max(1, num_iterations // 10) == 0:
478         print(f' {i+1:5d}/{num_iterations}  accept_rate={accept_count/(i+1):.3f}
479
480     if verbose:
481         print(f'HMC done: accept_rate={accept_count/num_iterations:.3f}  t={time()-t0:.
482     return jnp.array(thetas)
483
484
485     # --- Convergence diagnostics ---
486
487     def _gelman_rubin_variance(x: np.ndarray) -> float:
488         """Marginal posterior variance estimate (Gelman-Rubin). x shape: (num_chains, num_i
489         m, n = x.shape
490         B_over_n = ((jnp.mean(x, axis=1) - jnp.mean(x)) ** 2).sum() / (m - 1)
491         W = ((x - x.mean(axis=1, keepdims=True)) ** 2).sum() / (m * (n - 1))
492         return W * (n - 1) / n + B_over_n
493
494
495     def compute_Rhat(chains: ArrayLike) -> np.ndarray:
496         """Gelman-Rubin R-hat convergence diagnostic. Values near 1.0 indicate convergence.
497
498         arguments:
499             chains: array of shape (num_chains, num_samples, num_params)
500
501         returns:
502             Rhat: array of shape (num_params,)
503         """
504         chains = np.array(chains)
505         num_chains, num_samples, num_params = chains.shape
506
507         # split each chain in half to double the number of sub-chains
508         half = num_samples // 2
509         sub = np.concatenate([chains[:, :half, :], chains[:, half:2*half, :]], axis=0) # (
510         m, n = sub.shape[0], sub.shape[1]
511
512         chain_means = sub.mean(axis=1) # (m, n
513         chain_vars = ((sub - chain_means[:, None, :]) ** 2).sum(axis=1) / (n - 1) # (m, nu
514
515         global_mean = chain_means.mean(axis=0)
516         B = n / (m - 1) * ((chain_means - global_mean) ** 2).sum(axis=0)

```

```

517     W = chain_vars.mean(axis=0)
518
519     var_est = (n - 1) / n * W + B / n
520     return np.sqrt(var_est / W)
521
522
523 def compute_effective_sample_size(chains: ArrayLike) -> np.ndarray:
524     """Effective sample size (ESS) for each parameter.
525
526     arguments:
527         chains: array of shape (num_chains, num_samples, num_params)
528
529     returns:
530         ESS: array of shape (num_params,)
531     """
532     chains = np.array(chains)
533     num_chains, num_samples, num_params = chains.shape
534
535     def _ess_single(x: np.ndarray) -> int:
536         m, n = x.shape
537         post_var = _gelman_rubin_variance(x)
538         variogram = lambda t: ((x[:, t:] - x[:, :(n - t)]) ** 2).sum() / (m * (n - t))
539         rho = np.ones(n)
540         t, negative = 1, False
541         while not negative and t < n:
542             rho[t] = 1 - variogram(t) / (2 * post_var)
543             if t % 2 == 0:
544                 negative = rho[t - 1] + rho[t] < 0
545             t += 1
546         return int(m * n / (1 + 2 * rho[1:t].sum()))
547
548     return np.array([_ess_single(chains[:, :, p]) for p in range(num_params)])
549
550
551 # ===== General Minimal Plotting Functions =====
552
553 def _plot_data(ax: Axes, Xtrain: ArrayLike, ytrain: ArrayLike) -> None:
554     ax.plot(Xtrain, ytrain, 'k.', markersize=12, label='Data')
555     ax.grid(True)
556     ax.set_xlabel('Input $x$')
557     ax.set_ylabel('Response $y$')
558     ax.legend()
559
560 def plot_data(Xtrain: ArrayLike, ytrain: ArrayLike) -> None:
561     fig, ax = plt.subplots(1, 1, figsize=(10, 4))
562     _plot_data(ax, Xtrain, ytrain)
563
564 # Plot histogram

```

```
565
566 def plot_contour(ax: Axes,
567                 f: Callable[[ArrayLike, ArrayLike], jax.Array],
568                 x1s: ArrayLike,
569                 x2s: ArrayLike,
570                 num_contours: int = 10,
571                 transform: Callable[[ArrayLike], jax.Array] = lambda x: x,
572                 color: str = 'b',
573                 alpha: float = 1.0,
574                 xlabel: str = '$x_1$',
575                 ylabel: str = '$x_2$',
576                 title: str = '') -> None:
577     """Plot contour lines of a 2D function f(X1, X2).
578
579     arguments:
580         ax          -- matplotlib axes
581         f           -- callable f(X1, X2) where X1, X2 are NxM meshgrids; returns NxM
582         x1s         -- 1D array of values along the first axis (N points)
583         x2s         -- 1D array of values along the second axis (M points)
584         num_contours -- number of contour levels
585         transform    -- optional transform applied to f values before plotting (e.g. jnp
586         color       -- contour line color
587         alpha       -- contour line opacity
588         xlabel      -- x-axis label
589         ylabel      -- y-axis label
590         title       -- axes title
591     """
592     X1, X2 = jnp.meshgrid(x1s, x2s, indexing='ij')
593     Z = transform(f(X1, X2))
594     ax.contour(x1s, x2s, Z.T, num_contours, colors=color, alpha=alpha)
595     ax.set(xlabel=xlabel, ylabel=ylabel)
596     if title:
597         ax.set_title(title, fontweight='bold')
598
599 def plot_heatmap(fig: Figure,
600                 ax: Axes,
601                 f: Callable[[ArrayLike, ArrayLike], jax.Array],
602                 x1s: ArrayLike,
603                 x2s: ArrayLike,
604                 transform: Callable[[ArrayLike], jax.Array] = lambda x: x,
605                 xlabel: str = '$x_1$',
606                 ylabel: str = '$x_2$',
607                 title: str = '',
608                 cmap: str = 'viridis') -> None:
609     """Plot a 2D function f(X1, X2) as a heatmap with a colorbar on the right.
610
611     arguments:
612         fig          -- matplotlib figure (needed for the colorbar)
```

```

613         ax            -- matplotlib axes
614         f             -- callable f(X1, X2) where X1, X2 are NxM meshgrids; returns NxM
615         x1s           -- 1D array of values along the first axis (N points)
616         x2s           -- 1D array of values along the second axis (M points)
617         transform      -- optional transform applied to f values before plotting (e.g. jn
618         xlabel         -- x-axis label
619         ylabel         -- y-axis label
620         title          -- axes title
621         cmap           -- matplotlib colormap name
622     """
623     X1, X2 = jnp.meshgrid(x1s, x2s, indexing='ij')
624     Z = transform(f(X1, X2))
625     im = ax.pcolormesh(x1s, x2s, Z.T, shading='auto', cmap=cmap)
626     add_colorbar(im, fig, ax)
627     ax.set(xlabel=xlabel, ylabel=ylabel)
628     if title:
629         ax.set_title(title, fontweight='bold')
630
631 # ===== MCMC Plotting Functions =====
632
633 def plot_trace(axes: Union[Axes, Sequence[Axes]],
634               samples: ArrayLike,
635               param_names: Optional[list[str]] = None) -> None:
636     """Trace plots for each parameter.
637
638     arguments:
639         axes:          single axes (1 parameter) or array/list of axes (one per paramete
640         samples:       array of shape (num_samples, num_params) or (num_samples,) for 1D
641         param_names:   list of parameter name strings; defaults to  $\theta_0$ ,  $\theta_1$ , ...
642     """
643     s = jnp.asarray(samples)
644     s = jnp.atleast_2d(s) if s.ndim == 1 else s
645     num_params = s.shape[1] if s.ndim == 2 else 1
646     axes_list: list[Axes] = [axes] if isinstance(axes, Axes) else list(axes) # type: i
647     names = param_names or [f'$\\theta_{i}$' for i in range(num_params)]
648     for i, ax in enumerate(axes_list):
649         ax.plot(s[:, i] if s.ndim == 2 else s, lw=0.7)
650         ax.set(xlabel='Iteration', ylabel=names[i], title=f'Trace of {names[i]}')
651
652
653 def plot_mcmc_diagnostics(chains: ArrayLike,
654                          warm_up: int = 0,
655                          param_names: Optional[list[str]] = None,
656                          figsize: Optional[tuple[float, float]] = None,
657                          ) -> tuple[Figure, np.ndarray]:
658     """Full diagnostics panel: full trace | post-warmup trace | histogram, one row per
659     Titles include R-hat and ESS where applicable (requires  $\geq 2$  chains).
660

```

```

661     arguments:
662         chains:      array of shape (num_chains, num_samples, num_params)
663                     or (num_samples, num_params) for a single chain
664         warm_up:     number of warm-up samples to discard for diagnostics and histogram
665         param_names: list of parameter name strings
666         figsize:     figure size tuple; auto-scaled by default
667
668     returns:
669         fig, axes
670     """
671     chains = np.array(chains)
672     if chains.ndim == 2:
673         chains = chains[None, :, :]          # add chain dimension
674     num_chains, num_samples, num_params = chains.shape
675     names = param_names or [f'$\\theta_{i}$' for i in range(num_params)]
676
677     post_chains = chains[:, warm_up:, :]
678     multi = num_chains >= 2
679     Rhats = compute_Rhat(post_chains) if multi else np.full(num_params, np.nan)
680     ESSs = compute_effective_sample_size(post_chains) if multi else np.full(num_params,
681
682     fig, axes = plt.subplots(num_params, 3, figsize=figsize or (18, 3 * num_params), sq
683
684     for p in range(num_params):
685         n = names[p]
686
687         # full trace
688         axes[p, 0].plot(chains[:, :, p].T, lw=0.6, alpha=0.8)
689         if warm_up > 0:
690             axes[p, 0].axvline(warm_up, color='r', linestyle='--', lw=1, label='warm-up
691             axes[p, 0].set(xlabel='Iteration', ylabel=n, title=f'Full trace {n}')
692
693         # post-warmup trace
694         rhat_str = f' $\\hat{{R}}_{p}={Rhats[p]:.3f}$' if multi else ''
695         axes[p, 1].plot(post_chains[:, :, p].T, lw=0.6, alpha=0.8)
696         axes[p, 1].set(xlabel='Iteration', ylabel=n, title=f'Post-warmup trace {n}{rha
697
698         # histogram
699         ess_str = f' ESS={ESSs[p]:.0f}$' if multi else ''
700         flat = post_chains[:, :, p].ravel()
701         axes[p, 2].hist(flat, bins=40, density=True, color=colors[0], alpha=0.7)
702         lo, hi = np.percentile(flat, [2.5, 97.5])
703         axes[p, 2].axvline(lo, color='r', linestyle='--', lw=1, label='95% interval')
704         axes[p, 2].axvline(hi, color='r', linestyle='--', lw=1)
705         axes[p, 2].set(xlabel=n, title=f'Histogram {n}{ess_str}')
706         axes[p, 2].legend(fontsize=9)
707
708     fig.tight_layout()

```

```

709     return fig, axes
710
711
712 def plot_posterior_1d(ax: Axes,
713                      samples: ArrayLike,
714                      log_target: Optional[Callable[[ArrayLike], jax.Array]] = None,
715                      x_range: Optional[tuple[float, float]] = None,
716                      num_bins: int = 40,
717                      color: str = 'b',
718                      label: str = 'Posterior') -> None:
719     """Histogram of 1D samples with optional rescaled log_target overlay.
720
721     arguments:
722         ax:          matplotlib axes
723         samples:      1D array of posterior samples
724         log_target:   optional callable for the log unnormalized density; plotted as a r
725         x_range:      (lo, hi) tuple; defaults to sample range with 10% padding
726         num_bins:     number of histogram bins
727         color:        histogram color
728         label:        legend label for the histogram
729     """
730     samples = np.array(samples).ravel()
731     if x_range is None:
732         margin = 0.1 * (samples.max() - samples.min())
733         x_range = (float(samples.min() - margin), float(samples.max() + margin))
734     lo, hi = x_range
735     ax.hist(samples, bins=num_bins, density=True, color=color, alpha=0.5, label=label)
736     if log_target is not None:
737         xs = jnp.linspace(lo, hi, 500)
738         log_p = np.array([log_target(jnp.array([x])) for x in xs])
739         p = np.exp(log_p - log_p.max())
740         ax.plot(xs, p / np.trapezoid(p, xs), color=color, lw=2, label='Target (rescaled
741     ax.set(xlabel='$\\theta$', ylabel='Density')
742     ax.legend()
743
744
745 def _plot_interval(ax: Axes, x: ArrayLike, samples: ArrayLike,
746                    interval: float, color: str, alpha: float, **kwargs) -> None:
747     lo = np.percentile(samples, 0.5 * (100 - interval), axis=0)
748     hi = np.percentile(samples, 100 - 0.5 * (100 - interval), axis=0)
749     ax.fill_between(np.array(x).ravel(), lo, hi, color=color, alpha=alpha, **kwargs)
750
751
752 def plot_predictions(ax: Axes,
753                    x: ArrayLike,
754                    samples: ArrayLike,
755                    num_samples: int = 100,
756                    sample_color: str = 'k',

```



```

757         sample_alpha: float = 0.3,
758         color: str = 'r',
759         legend: bool = False,
760         plot_mean: bool = True,
761         seed: int = 123,
762         title: str = '') -> None:
763     """Layered credibility-interval plot for posterior predictive samples.
764     Plots 99%, 95%, and 75% intervals as shaded bands plus individual sample curves.
765
766     arguments:
767         ax:          matplotlib axes
768         x:            1D array of input values for the x-axis
769         samples:      2D array of shape (num_samples, num_x_points)
770         num_samples:  number of individual sample curves to overlay
771         sample_color: color for individual sample curves
772         sample_alpha: opacity for individual sample curves
773         color:        color for interval bands and mean
774         legend:       whether to show legend
775         plot_mean:    whether to overlay the posterior mean
776         seed:         random seed for selecting sample curves
777         title:        axes title
778     """
779     samples = np.array(samples)
780     x = np.array(x).ravel()
781
782     for interval, alpha in [(99, 0.20), (95, 0.30), (75, 0.50)]:
783         _plot_interval(ax, x, samples, interval, color, alpha,
784                       label=f'{interval}% interval' if interval == 95 else None)
785
786     if num_samples > 0:
787         np.random.seed(seed)
788         idx = np.random.choice(len(samples), size=min(num_samples, len(samples)), repla
789                               ax.plot(x, samples[idx].T, color=sample_color, alpha=sample_alpha, lw=0.5)
790
791     if plot_mean:
792         ax.plot(x, samples.mean(axis=0), '-', color='k', lw=2.5, label='Mean')
793
794     if title:
795         ax.set_title(title, fontweight='bold')
796     if legend:
797         ax.legend(loc='best')
798
799
800     # ===== Gaussian Processes =====
801
802     def generate_samples(key: jax.Array, m: ArrayLike, K: ArrayLike,
803                        num_samples: int, jitter: float = 0) -> jax.Array:
804         """ returns M samples from an Gaussian process with mean m and kernel matrix K. The

```

```

805
806
807     arguments:
808         key                -- jax random key for controlling the random number generator
809         m                  -- mean vector (shape (N,))
810         K                  -- kernel matrix (shape NxN)
811         num_samples        -- number of samples to generate (positive integer)
812         jitter             -- amount of jitter (non-negative scalar)
813
814     returns
815         f_samples          -- a numpy matrix containing the samples of f (shape N x num_s
816     """
817
818     zs = random.normal(key, shape=(len(K), num_samples))
819     N = len(K)
820     L = jnp.linalg.cholesky(K + jitter*jnp.identity(N))
821     f_samples = m[:, None] + jnp.dot(L, zs)
822     assert f_samples.shape == (len(K), num_samples), f"The shape of f_samples appears w
823     return f_samples
824
825 @dataclass
826 class Hyperparameters(object):
827     kappa:          float = 1.0 # magnitude, positive scalar (default=1.0)
828     lengthscale:    float = 1.0 # characteristic lengthscale, positive scalar (default=
829     sigma:          float = 1.0 # noise std. dev., positive scalar (default=1.0)
830
831     def to_array(self) -> jax.Array:
832         """ return hyperparameters as flat JaX-array (to be used later) """
833         return jnp.array([self.kappa, self.lengthscale, self.sigma])
834
835     @staticmethod
836     def from_array(hyper_array: jax.Array) -> Hyperparameters:
837         """ instantiates Hyperparameter object from flat JaX-array (or list) of hyperpa
838         kappa, lengthscale, sigma = hyper_array
839         return Hyperparameters(kappa, lengthscale, sigma)
840
841     def __repr__(self) -> str:
842         """ for reporting hyperparameter values """
843         return f'Hyperparameters(kappa={self.kappa:3.2f}, lengthscale={self.lengthscale
844
845 # in the code below tau represents the distance between to input points, i.e. tau = ||x
846 def squared_exponential(tau: ArrayLike, hyperparameters: Hyperparameters) -> jax.Array:
847     return hyperparameters.kappa**2*jnp.exp(-0.5*tau**2/hyperparameters.lengthscale**2)
848
849 def matern12(tau: ArrayLike, hyperparameters: Hyperparameters) -> jax.Array:
850     return hyperparameters.kappa**2*jnp.exp(-tau/hyperparameters.lengthscale)
851
852 def matern32(tau: ArrayLike, hyperparameters: Hyperparameters) -> jax.Array:

```

```

853     return hyperparameters.kappa**2*(1 + jnp.sqrt(3)*tau/hyperparameters.lengthscale)*j
854
855 class StationaryIsotropicKernel(object):
856
857     def __init__(self, kernel_fun: Callable[[ArrayLike, Hyperparameters], jax.Array]) -
858         """
859         the argument kernel_fun must be a function of two arguments kernel_fun(||ta
860         squared_exponential = lambda tau, hyper: hyper.kappa**2*np.exp(-0.5*tau**2/
861         """
862         self.kernel_fun = kernel_fun
863
864     def construct_kernel(self, X1: ArrayLike, X2: ArrayLike,
865                         hyperparameters: Hyperparameters, jitter: float = 1e-8) -> jax
866         """ compute and returns the NxM kernel matrix between the two sets of input X1
867
868         arguments:
869             X1          -- NxD matrix
870             X2          -- MxD matrix or None
871             hyperparameters -- Hyperparameter object compatible with self.kernel_fun fu
872             jitter      -- non-negative scalar
873
874         returns
875             K          -- NxM matrix
876         """
877
878         N, M = X1.shape[0], X2.shape[0]
879         dists = jnp.sqrt(jnp.sum((jnp.expand_dims(X1, 1) - jnp.expand_dims(X2, 0))**2,
880         K = self.kernel_fun(dists, hyperparameters)
881
882         if len(X1) == len(X2) and jnp.allclose(X1, X2):
883             K = K + jitter*jnp.identity(len(X1))
884         assert K.shape == (N, M), f"The shape of K appears wrong. Expected shape ({N},
885         return K
886
887 class Kernel(object):
888     """General kernel, not restricted to stationary or isotropic functions.
889
890     The kernel_fun must be a function of (x1, x2, hyperparameters) where x1 and x2
891     are individual D-dimensional input vectors and the return value is a scalar.
892
893     Example – linear kernel:
894         Kernel(lambda x1, x2, hyper: hyper.kappa**2 * jnp.dot(x1, x2))
895
896     Example – polynomial kernel of degree p:
897         Kernel(lambda x1, x2, hyper: (hyper.kappa + jnp.dot(x1, x2))**hyper.lengthscale
898     """
899
900     def __init__(self, kernel_fun: Callable[[ArrayLike, ArrayLike, Hyperparameters], ja

```

```

901         """
902         arguments:
903             kernel_fun -- function kernel_fun(x1, x2, hyperparameters) where
904                          x1 (shape (D,)) and x2 (shape (D,)) are input vectors.
905                          Must return a scalar kernel value k(x1, x2).
906         """
907         self.kernel_fun = kernel_fun
908
909     def construct_kernel(self, X1: ArrayLike, X2: ArrayLike,
910                         hyperparameters: Hyperparameters, jitter: float = 1e-8) -> jax.Array:
911         """Compute and return the NxM kernel matrix between X1 (NxD) and X2 (MxD).
912
913         arguments:
914             X1 -- NxD matrix
915             X2 -- MxD matrix
916             hyperparameters -- Hyperparameter object compatible with self.kernel_fun
917             jitter -- non-negative scalar, added to diagonal when X1 == X2
918
919         returns
920             K -- NxM matrix
921         """
922         N, M = X1.shape[0], X2.shape[0]
923
924         K = vmap(lambda x1: vmap(lambda x2: self.kernel_fun(x1, x2, hyperparameters)))(X1, X2)
925
926         if len(X1) == len(X2) and jnp.allclose(X1, X2):
927             K = K + jitter * jnp.identity(N)
928         assert K.shape == (N, M), f"The shape of K appears wrong. Expected shape ({N}, {M})."
929         return K
930
931
932     class GaussianProcessRegression(object):
933
934     def __init__(self, X: ArrayLike, y: ArrayLike,
935                 kernel: StationaryIsotropicKernel | Kernel,
936                 hyperparameters: Hyperparameters,
937                 jitter: float = 1e-8) -> None:
938         """
939         Arguments:
940             X -- NxD input points
941             y -- Nx1 observed values
942             kernel -- must be instance of StationaryIsotropicKernel or Kernel
943             jitter -- non-negative scalar
944             hyperparameters -- Hyperparameter object containing kernel hyperparameters
945         """
946         self.X = X
947         self.y = y
948         self.N = len(X)

```

```
949         self.kernel = kernel
950         self.jitter = jitter
951         self.set_hyperparameters(hyperparameters)
952         self.check_dimensions()
953
954     def check_dimensions(self) -> None:
955         N, D = self.X.shape
956         assert self.X.ndim == 2, f"The variable X must be of shape (N, D), however, the
957         assert self.y.ndim == 2, f"The variable y must be of shape (N, 1), however. the
958         assert self.y.shape == (N, 1), f"The variable y must be of shape (N, 1), however
959
960     def set_hyperparameters(self, hyper: Hyperparameters) -> None:
961         self.hyperparameters = hyper
962
963     def posterior_samples(self, key: jax.Array, Xstar: ArrayLike, num_samples: int) ->
964         """
965         generate samples from the posterior  $p(f^*|y, x^*)$  for each of the inputs in
966
967         Arguments:
968             key          -- jax random key for controlling the random number ge
969             Xstar        -- PxD prediction points
970
971         returns:
972             f_samples    -- numpy array of (P, num_samples) containing num_samp
973         """
974
975         mu, Sigma = self.predict_f(Xstar)
976         f_samples = generate_samples(key, mu.ravel(), Sigma, num_samples)
977
978         assert (f_samples.shape == (len(Xstar), num_samples)), f"The shape of the poste
979         return f_samples
980
981     def predict_y(self, Xstar: ArrayLike) -> tuple[jax.Array, jax.Array]:
982         """ returns the posterior distribution of  $y^*$  evaluated at each of the points i
983
984         Arguments:
985             Xstar        -- PxD prediction points
986
987         returns:
988             mu           -- Px1 mean vector
989             Sigma        -- PxP covariance matrix
990         """
991
992         mu, Sigma = self.predict_f(Xstar)
993         Sigma = Sigma + self.hyperparameters.sigma**2 * jnp.identity(len(mu))
994
995         return mu, Sigma
996
```

```

997     def predict_f(self, Xstar: ArrayLike) -> tuple[jax.Array, jax.Array]:
998         """ returns the posterior distribution of  $f^*$  evaluated at each of the points i
999
1000         Arguments:
1001         Xstar          -- PxD prediction points
1002
1003         returns:
1004         mu              -- Px1 mean vector
1005         Sigma           -- PxP covariance matrix
1006         """
1007
1008         k = self.kernel.construct_kernel(Xstar, self.X, self.hyperparameters, jitter=se
1009         K = self.kernel.construct_kernel(self.X, self.X, self.hyperparameters, jitter=s
1010         Kstar = self.kernel.construct_kernel(Xstar, Xstar, self.hyperparameters, jitter
1011
1012         C = K + self.hyperparameters.sigma**2*jnp.identity(len(self.X))
1013
1014         mu = jnp.dot(k, jnp.linalg.solve(C, self.y))
1015         Sigma = Kstar - jnp.dot(k, jnp.linalg.solve(C, k.T))
1016
1017         assert (mu.shape == (len(Xstar), 1)), f"The shape of the posterior mu seems wro
1018         assert (Sigma.shape == (len(Xstar), len(Xstar))), f"The shape of the posterior
1019
1020         return mu, Sigma
1021
1022     def log_marginal_likelihood(self, hyperparameters: Hyperparameters) -> jax.Array:
1023         """
1024             evaluate the log marginal likelihood  $p(y)$  given the hyperparaemters
1025
1026         Arguments:
1027             hyperparameters -- Hyperparameter object containing kernel hyperparame
1028         """
1029
1030         K = self.kernel.construct_kernel(self.X, self.X, hyperparameters)
1031         C = K + hyperparameters.sigma**2*jnp.identity(self.N)
1032
1033         L = jnp.linalg.cholesky(C)
1034         v = jnp.linalg.solve(L, self.y)
1035
1036         logdet_term = jnp.sum(jnp.log(jnp.diag(L)))
1037         quad_term = 0.5*jnp.sum(v**2)
1038         const_term = -0.5*self.N*jnp.log(2*jnp.pi)
1039
1040         return const_term - logdet_term - quad_term
1041
1042     def optimize_marginal_likelihood(gp: GaussianProcessRegression,
1043                                     hyperparameters_init: Hyperparameters,
1044                                     verbose: bool = True) -> Hyperparameters:

```

```

1045     """ Optimize log marginal likelihood with gradient-based methods """
1046
1047     def objective(log_hyperparam_array: jax.Array) -> jax.Array:
1048         hyperparams_array = jnp.exp(log_hyperparam_array)
1049         hyper = Hyperparameters.from_array(hyperparams_array)
1050         return -gp.log_marginal_likelihood(hyper)
1051
1052     log_hyper_array = jnp.log(hyperparameters_init.to_array())
1053     res = minimize(value_and_grad(objective), log_hyper_array, jac=True)
1054
1055     if not res.success:
1056         print('Warning: optimization failed!')
1057
1058     log_hyper_hat = res.x
1059     hyper = Hyperparameters.from_array(jnp.exp(log_hyper_hat))
1060
1061     if verbose:
1062         print('Result of optimization:', hyper)
1063
1064     return hyper
1065
1066 # ===== Gaussian Process Plotting Functions =====
1067
1068 def add_colorbar(im, fig: Figure, ax: Axes) -> None:
1069     divider = make_axes_locatable(ax)
1070     cax = divider.append_axes('right', size='5%', pad=0.05)
1071     fig.colorbar(im, cax=cax, orientation='vertical')
1072
1073 def plot_kernel(X: ArrayLike, K: ArrayLike, hyper: Hyperparameters,
1074                 key: jax.Array, num_samples: int) -> None:
1075     fig, ax = plt.subplots(1, 2, figsize=(20, 5))
1076
1077     im = ax[0].pcolormesh(X.flatten(), X.flatten(), K, shading='auto')
1078     ax[0].set(xlabel='Input $x$', ylabel="Input $x'$", title=f"Kernel function $k(x, x'$")
1079     ax[0].grid(False)
1080     ax[0].set_aspect('equal')
1081     add_colorbar(im, fig, ax[0])
1082
1083     m = jnp.zeros(len(X))
1084     f_samples = generate_samples(key, m, K, num_samples=num_samples, jitter=1e-8)
1085     ax[1].plot(X, f_samples, alpha=0.75, linewidth=3);
1086     ax[1].grid(True)
1087     ax[1].set(xlabel='$x$', ylabel='$f(x)$', title='Samples from the Gaussian process')
1088
1089
1090 def _plot_with_uncertainty(ax: Axes, Xp: ArrayLike, gp: GaussianProcessRegression,
1091                           color: str = 'r', color_samples: str = 'b',
1092                           title: str = "", num_samples: int = 0, seed: int = 0) -> Non

```

```

1093
1094 mu, Sigma = gp.predict_y(Xp)
1095 mean, std = mu.ravel(), jnp.sqrt(jnp.diag(Sigma))
1096
1097 key = random.PRNGKey(seed)
1098
1099 ax.plot(Xp, mean, color=color, label='Mean')
1100 ax.plot(Xp, mean + 2*std, color=color, linestyle='--')
1101 ax.plot(Xp, mean - 2*std, color=color, linestyle='--')
1102 ax.fill_between(Xp.ravel(), mean - 2*std, mean + 2*std, color=color, alpha=0.25, la
1103
1104 if num_samples > 0:
1105     fs = gp.posterior_samples(key, Xp, num_samples)
1106     ax.plot(Xp, fs[:,0], color=color_samples, alpha=.25, label="$f(x)$ samples")
1107     ax.plot(Xp, fs[:, 1:], color=color_samples, alpha=.25)
1108 ax.set_title(title)
1109
1110 def plot_with_uncertainty(kernel: StationaryIsotropicKernel | Kernel,
1111                          hyper: Hyperparameters,
1112                          Xtrain: ArrayLike,
1113                          ytrain: ArrayLike,
1114                          Xstar: ArrayLike) -> None:
1115     gp_prior = GaussianProcessRegression(jnp.zeros((0, 1)), jnp.zeros((0, 1)), kernel,
1116     gp_post = GaussianProcessRegression(Xtrain, ytrain, kernel, hyper)
1117     fig, ax = plt.subplots(1, 2, figsize=(25, 6))
1118     _plot_with_uncertainty(ax[0], Xstar, gp_prior, title='Prior predictive distribution
1119     _plot_with_uncertainty(ax[1], Xstar, gp_post, title='Posterior predictive distribut
1120     for i in range(2):
1121         _plot_data(ax[i], Xtrain, ytrain)
1122         ax[i].legend(loc='lower center', ncol=4)
1123
1124
1125 def plot_with_uncertainty_laplace(kernel: StationaryIsotropicKernel | Kernel,
1126                                  hyper: Hyperparameters,
1127                                  X: ArrayLike,
1128                                  Xstar: ArrayLike,
1129                                  m: ArrayLike,
1130                                  S: ArrayLike,
1131                                  ax: Optional[Axes] = None,
1132                                  color: str = 'r',
1133                                  title: str = '',
1134                                  ) -> tuple[jax.Array, jax.Array]:
1135     """GP predictive distribution using a Laplace-approximated posterior over f.
1136
1137     Given the Laplace posterior  $N(f \mid m, S)$  at training inputs  $X$ , computes
1138     the marginal predictive distribution of the latent function  $f^*$  at  $Xstar$ :
1139
1140     
$$\mu_* = K_{\{X\}} K_{\{XX\}}^{-1} m$$


```



```

1141     Sigma_* = K_{**} - K_{*X} K_{XX}^{-1} K_{X*}
1142               + K_{*X} K_{XX}^{-1} S K_{XX}^{-1} K_{X*}
1143
1144     This is the correct predictive distribution for non-conjugate GP models
1145     such as GP classification, where the Laplace approximation is applied to
1146     the intractable posterior over the latent function.
1147
1148     arguments:
1149         kernel    -- kernel object (StationaryIsotropicKernel or Kernel)
1150         hyper     -- Hyperparameters
1151         X         -- training inputs, shape (N, D)
1152         Xstar     -- test inputs, shape (P, D)
1153         m         -- Laplace MAP estimate of f at X, shape (N,) or (N, 1)
1154         S         -- Laplace posterior covariance at X, shape (N, N)
1155         ax        -- matplotlib axes; creates new figure if None
1156         color     -- color for the mean curve and interval
1157         title     -- axes title
1158
1159     returns:
1160         mu_star, std_star -- predictive mean (P,) and std (P,) at Xstar
1161     """
1162     X, Xstar = jnp.asarray(X), jnp.asarray(Xstar)
1163     m = jnp.asarray(m).ravel()
1164     S = jnp.asarray(S)
1165
1166     K_XX = kernel.construct_kernel(X, X, hyper)
1167     K_sX = kernel.construct_kernel(Xstar, X, hyper)
1168     K_ss = kernel.construct_kernel(Xstar, Xstar, hyper)
1169
1170     # mu_* = K_{*X} K_{XX}^{-1} m
1171     alpha = jnp.linalg.solve(K_XX, m)
1172     mu_star = K_sX @ alpha
1173
1174     # Sigma_* = K_{**} - K_{*X} V + K_{*X} K_{XX}^{-1} S V, V = K_{XX}^{-1} K_{X*}
1175     V = jnp.linalg.solve(K_XX, K_sX.T) # (N, P)
1176     Sigma_star = K_ss - K_sX @ V + K_sX @ jnp.linalg.solve(K_XX, S @ V)
1177     std_star = jnp.sqrt(jnp.maximum(jnp.diag(Sigma_star), 0.0))
1178
1179     if ax is None:
1180         _, ax = plt.subplots(figsize=(10, 4))
1181
1182     xs = Xstar.ravel()
1183     ax.plot(xs, mu_star, color=color, label='Predictive mean')
1184     ax.plot(xs, mu_star + 2 * std_star, '--', color=color, lw=0.8)
1185     ax.plot(xs, mu_star - 2 * std_star, '--', color=color, lw=0.8)
1186     ax.fill_between(xs, mu_star - 2 * std_star, mu_star + 2 * std_star,
1187                    color=color, alpha=0.25, label='95% interval')
1188     ax.scatter(X.ravel(), m, c='k', s=25, zorder=5, label='Training MAP $\hat{f}$')

```

```

1189         if title:
1190             ax.set_title(title, fontweight='bold')
1191         ax.legend()
1192
1193         return mu_star, std_star
1194
1195
1196 # ===== Variational Inference =====
1197
1198 from .exercise10 import VariationalGMM, plot_std_dev_contour, PCA_dim_reduction # noqa
1199 from .exercise11 import AdamOptimizer, create_linear_regression_data # noqa
1200
1201
1202 def kl_gaussian(m_q: ArrayLike, v_q: ArrayLike,
1203                 m_p: ArrayLike, v_p: ArrayLike) -> jax.Array:
1204     """Analytical KL(q||p) for diagonal Gaussians q = N(m_q, diag(v_q)), p = N(m_p, dia
1205
1206     arguments:
1207         m_q, v_q -- mean and variance vectors of q (shape (D,))
1208         m_p, v_p -- mean and variance vectors of p (shape (D,))
1209
1210     returns:
1211         kl -- KL divergence scalar
1212     """
1213     m_q, v_q, m_p, v_p = jnp.asarray(m_q), jnp.asarray(v_q), jnp.asarray(m_p), jnp.asar
1214     return 0.5 * jnp.sum(v_q / v_p + (m_q - m_p) ** 2 / v_p - 1.0 + jnp.log(v_p / v_q))
1215
1216
1217 class BlackBoxVI:
1218     """Black-Box Variational Inference with mean-field Gaussian approximation.
1219
1220     Optimizes the ELBO via the reparametrization trick and Adam.
1221     The variational family is  $q(w) = \prod_d N(w_d | m_d, v_d)$ .
1222     Parameters are stored in unconstrained form:  $\text{lam} = [m, \log(v)]$ .
1223
1224     Usage::
1225         # log_prior(w): w shape (S, D) -> (S,)
1226         # log_lik(X, y, w): w shape (S, D) -> (S,)
1227         log_prior = lambda w: jnp.sum(gaussian_logpdf(w, 0., 1.), axis=-1)
1228         log_lik = lambda X, y, w: jnp.sum(bernoulli_logpmf(y, sigmoid(w @ X.T)), axis
1229         vi = BlackBoxVI(log_prior, log_lik, num_params=D)
1230         vi.fit(X, y)
1231         samples = vi.generate_posterior_samples(key, num_samples=500)
1232     """
1233
1234     def __init__(self,
1235                 log_prior: Callable[[ArrayLike], jax.Array],
1236                 log_lik: Callable[[ArrayLike, ArrayLike, ArrayLike], jax.Array],

```

```

1237         num_params: int,
1238         step_size: float = 0.01,
1239         max_itt: int = 1000,
1240         num_samples: int = 10,
1241         batch_size: Optional[int] = None,
1242         seed: int = 0,
1243         verbose: int = 200) -> None:
1244     """
1245     arguments:
1246         log_prior    -- callable log_prior(w) where w has shape (S, D); returns (S,)
1247         log_lik       -- callable log_lik(X, y, w) where w has shape (S, D); returns
1248         num_params    -- D, dimension of the parameter space
1249         step_size     -- Adam step size
1250         max_itt       -- number of Adam iterations
1251         num_samples   -- number of MC samples S for the ELBO estimate
1252         batch_size    -- mini-batch size (None = full data)
1253         seed          -- random seed
1254         verbose       -- print ELBO every `verbose` iterations (0 = silent)
1255     """
1256     self.log_prior = log_prior
1257     self.log_lik = log_lik
1258     self.D = num_params
1259     self.step_size = step_size
1260     self.max_itt = max_itt
1261     self.num_samples = num_samples
1262     self.batch_size = batch_size
1263     self.seed = seed
1264     self.verbose = verbose
1265
1266     m0 = jnp.zeros(self.D)
1267     v0 = jnp.ones(self.D)
1268     self.lam = self.pack(m0, v0)
1269
1270     self.ELBO_history: list[float] = []
1271     self.m_history: list[np.ndarray] = []
1272     self.v_history: list[np.ndarray] = []
1273
1274     def pack(self, m: ArrayLike, v: ArrayLike) -> jax.Array:
1275         """Pack (m, v) into unconstrained lam = [m, log(v)]."""
1276         return jnp.concatenate([m, jnp.log(v)])
1277
1278     def unpack(self, lam: ArrayLike) -> tuple[jax.Array, jax.Array]:
1279         """Unpack lam into (m, v) with v = exp(lam[D:])."""
1280         lam = jnp.asarray(lam)
1281         return lam[:self.D], jnp.exp(lam[self.D:])
1282
1283     def compute_entropy(self, v: ArrayLike) -> jax.Array:
1284         """Entropy of mean-field Gaussian: sum_d 0.5*(log(2*pi*v_d) + 1)."""

```

```
1285         return 0.5 * jnp.sum(jnp.log(2 * jnp.pi * v) + 1.0)
1286
1287     def compute_ELBO(self, lam: ArrayLike, key: jax.Array,
1288                     X: Optional[ArrayLike] = None,
1289                     y: Optional[ArrayLike] = None) -> jax.Array:
1290         """MC-ELBO estimate via reparametrization:  $E_q[\log p(w) + \log p(y|w)] + H[q]$ .
1291
1292         arguments:
1293             lam -- unconstrained variational parameters, shape (2*D,)
1294             key -- JAX random key
1295             X -- input data passed to log_lik (may be a mini-batch)
1296             y -- target data passed to log_lik (may be a mini-batch)
1297
1298         returns:
1299             elbo -- scalar ELBO estimate
1300         """
1301         m, v = self.unpack(lam)
1302         eps = random.normal(key, shape=(self.num_samples, self.D))
1303         w = m[None, :] + jnp.sqrt(v)[None, :] * eps # (S, D) reparametrized
1304         log_prior = jnp.mean(self.log_prior(w))
1305         log_lik: jax.Array | float = 0.0
1306         if X is not None and y is not None:
1307             log_lik = jnp.mean(self.log_lik(X, y, w))
1308         return log_prior + log_lik + self.compute_entropy(v)
1309
1310     def generate_posterior_samples(self, key: jax.Array, num_samples: int) -> jax.Array
1311         """Draw samples from the fitted variational posterior  $q(w)$ .
1312
1313         returns:
1314             samples -- shape (num_samples, D)
1315         """
1316         m, v = self.unpack(self.lam)
1317         eps = random.normal(key, shape=(num_samples, self.D))
1318         return m[None, :] + jnp.sqrt(v)[None, :] * eps
1319
1320     def fit(self, X: ArrayLike, y: ArrayLike) -> BlackBoxVI:
1321         """Optimize the ELBO with Adam.
1322
1323         arguments:
1324             X -- input data, shape (N, ...)
1325             y -- targets, shape (N, ...)
1326
1327         returns:
1328             self
1329         """
1330         X, y = jnp.asarray(X), jnp.asarray(y)
1331         N = len(X)
1332         key = random.PRNGKey(self.seed)
```

```
1333     lam = self.lam
1334     optimizer = AdamOptimizer(len(lam), lam, self.step_size)
1335     elbo_grad = jit(value_and_grad(self.compute_ELBO))
1336
1337     for i in range(self.max_itt):
1338         key, key_elbo, key_batch = random.split(key, 3)
1339         if self.batch_size is not None:
1340             idx = random.choice(key_batch, N, shape=(self.batch_size,), replace=False)
1341             Xb, yb = X[idx], y[idx]
1342         else:
1343             Xb, yb = X, y
1344
1345         elbo, g = elbo_grad(lam, key_elbo, Xb, yb)
1346         lam = optimizer.step(g)
1347
1348         self.ELBO_history.append(float(elbo))
1349         m, v = self.unpack(lam)
1350         self.m_history.append(np.array(m))
1351         self.v_history.append(np.array(v))
1352
1353         if self.verbose and (i + 1) % self.verbose == 0:
1354             print(f' {i+1:5d}/{self.max_itt} ELBO={elbo:.3f}')
1355
1356     self.lam = lam
1357     return self
1358
1359
1360 # ===== Variational Inference Plotting =====
1361
1362 def plot_elbo(ax: Axes, vi: BlackBoxVI,
1363              color: str = 'b', label: str = 'ELBO') -> None:
1364     """Plot ELBO convergence history.
1365
1366     arguments:
1367         ax      -- matplotlib axes
1368         vi      -- fitted BlackBoxVI instance
1369         color   -- line color
1370         label   -- legend label
1371
1372     """
1373     ax.plot(vi.ELBO_history, color=color, label=label)
1374     ax.set(xlabel='Iteration', ylabel='ELBO', title='ELBO Convergence')
1375     ax.legend()
1376
1377 def plot_vi_diagnostics(vi: BlackBoxVI,
1378                        param_names: Optional[list[str]] = None,
1379                        figsize: Optional[tuple[float, float]] = None,
1380                        ) -> tuple[Figure, np.ndarray]:
```

```
1381     """Diagnostic panel: ELBO history + variational mean  $\pm$  2 std per parameter.
1382
1383     arguments:
1384         vi            -- fitted BlackBoxVI instance
1385         param_names   -- list of parameter name strings; defaults to  $\theta_0, \theta_1, \dots$ 
1386         figsize       -- figure size
1387
1388     returns:
1389         fig, axes     -- (D+1 rows, 1 column)
1390     """
1391     D = vi.D
1392     names = param_names or [f'$\\theta_{i}$' for i in range(D)]
1393     m_hist = np.array(vi.m_history) # (num_iters, D)
1394     v_hist = np.array(vi.v_history) # (num_iters, D)
1395     iters = np.arange(len(m_hist))
1396
1397     fig, axes = plt.subplots(D + 1, 1, figsize=figsize or (10, 3 * (D + 1)), squeeze=False)
1398
1399     axes[0, 0].plot(vi.ELBO_history, color='b')
1400     axes[0, 0].set(xlabel='Iteration', ylabel='ELBO', title='ELBO Convergence')
1401
1402     for i in range(D):
1403         ax = axes[i + 1, 0]
1404         mean_i = m_hist[:, i]
1405         std_i = np.sqrt(v_hist[:, i])
1406         ax.plot(iters, mean_i, color=colors[0], label='Mean')
1407         ax.fill_between(iters, mean_i - 2 * std_i, mean_i + 2 * std_i,
1408                        color=colors[0], alpha=0.3, label='±2 std')
1409         ax.set(xlabel='Iteration', ylabel=names[i],
1410               title=f'Variational mean  $\pm$  2 std {names[i]}')
1411         ax.legend(fontsize=9)
1412
1413     fig.tight_layout()
1414     return fig, axes
```